

LAPORAN PRAKTIKUM STRUKTUR DATA

SORTING

Disusun Oleh:

Endy Pardilian 2511531017

Dosen Pengampu:

Wahyudi. Dr., S.T,M.T

Asisten Pratikum:

M Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas praktikum Struktur Data serta untuk memahami penerapan algoritma sorting dalam pemrograman Java.

Pada praktikum ini dibahas beberapa algoritma pengurutan data yaitu Shell Sort, Quick Sort, dan Merge Sort. Selain mempelajari konsep masing-masing algoritma, praktikan juga mengimplementasikan algoritma tersebut ke dalam program Java untuk memahami cara kerja dan perbedaannya dalam mengurutkan data.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan laporan di masa yang akan datang. Semoga laporan ini dapat memberikan manfaat bagi pembaca dan menambah wawasan mengenai algoritma sorting.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat.....	1
BAB II PEMBAHASAN.....	2
2.1 shellSort_2511531017.....	2
2.2 QuickSort_2511531017.....	4
2.3 MergeSort_2511531017.....	7
2.4 BubleGUI_2511531017.....	11
TUGAS.....	27
2.5 Lagu_2511531017.....	27
2.6 Sorting_2511531017.....	27
BAB III KESIMPULAN.....	32
3.1 Kesimpulan.....	32
3.2 Saran.....	32
DAFTAR PUSTAKA.....	33

BAB I

PENDAHULUAN

1.1 Latar Belakang

Sorting merupakan proses pengurutan data berdasarkan aturan tertentu, baik secara ascending maupun descending. Dalam ilmu komputer, proses sorting sangat penting karena dapat mempermudah pencarian, pengolahan, dan pengelolaan data sehingga lebih efisien.

Terdapat berbagai algoritma sorting yang memiliki karakteristik dan tingkat efisiensi yang berbeda-beda. Pada praktikum ini digunakan algoritma Shell Sort, Quick Sort, dan Merge Sort yang termasuk algoritma pengurutan yang lebih efisien dibandingkan metode sorting sederhana seperti Bubble Sort, Selection Sort, dan Insertion Sort.

Melalui implementasi algoritma-algoritma tersebut menggunakan bahasa pemrograman Java, praktikan dapat memahami cara kerja masing-masing metode serta mengetahui kelebihan dan kekurangan setiap algoritma dalam menyelesaikan proses pengurutan data.

1.2 Tujuan

1. Memahami konsep dan cara kerja algoritma Shell Sort, Quick Sort, dan Merge Sort.
2. Mengimplementasikan algoritma sorting menggunakan bahasa pemrograman Java.
3. Menganalisis hasil pengurutan data serta membandingkan karakteristik masing-masing algoritma sorting.

1.3 Manfaat

1. Menambah pemahaman mengenai berbagai algoritma pengurutan data yang digunakan dalam struktur data.
2. Meningkatkan kemampuan pemrograman Java melalui implementasi algoritma Shell Sort, Quick Sort, dan Merge Sort.
3. Memberikan pengalaman dalam menganalisis efisiensi dan penerapan algoritma sorting pada berbagai jenis data.

BAB II

PEMBAHASAN

2.1 shellSort_2511531017

```
package pekan8_2511531017;

public class shellSort_2511531017 {
    public static void shellShort_2511531017(int[] A_1017) {
        int n_1017 = A_1017.length;
        int gap_1017 = n_1017 / 2;
        while (gap_1017 > 0) {
            for (int i_1017 = gap_1017; i_1017 < n_1017; i_1017++) {
                int temp_1017 = A_1017[i_1017];
                int j_1017 = i_1017;
                while (j_1017 >= gap_1017 && A_1017[j_1017 - gap_1017] > temp_1017) {
                    A_1017[j_1017] = A_1017[j_1017 - gap_1017];
                    j_1017 = j_1017 - gap_1017;
                }
                A_1017[j_1017] = temp_1017;
            }
            gap_1017 = gap_1017 / 2;
        }
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] data_1017 = {3,10,4,6,8,9,7,2,1,5};
        System.out.print("Sebelum: ");
        printArray_1017(data_1017);
        shellShort_2511531017(data_1017);
        System.out.print("Sesudah (Shell Sort): ");
        printArray_1017(data_1017);
    }

    public static void printArray_1017(int[] arr_1017) {
        for (int i_1017 : arr_1017) System.out.print(i_1017 + " ");
        System.out.println();
    }
}
```

Kode 2.1

Penjelasan:

```
public static void shellShort_2511531017(int[] A_1017) {
    int n_1017 = A_1017.length;
    int gap_1017 = n_1017 / 2;
```

Membuat method Shell Sort yang menerima parameter berupa array integer. Variabel `n_1017` digunakan untuk menyimpan panjang array, sedangkan `gap_1017` digunakan sebagai jarak antar elemen yang akan dibandingkan. Nilai gap awal ditentukan sebesar setengah dari panjang array.

```
while (gap_1017 > 0) {
```

Setiap iterasi akan mengurangi nilai gap hingga akhirnya bernilai satu dan seluruh data menjadi terurut. Perulangan ini merupakan bagian utama dari algoritma Shell Sort.

```
for (int i_1017 = gap_1017; i_1017 < n_1017; i_1017++) {
    int temp_1017 = A_1017[i_1017];
    int j_1017 = i_1017;
```

Perulangan pada setiap elemen array sesuai jarak gap yang digunakan. Nilai elemen yang sedang diproses disimpan ke dalam variabel temp_1017. Variabel j_1017 digunakan sebagai indeks pembandingan saat proses penyisipan data.

```
while (j_1017 >= gap_1017 && A_1017[j_1017 - gap_1017] > temp_1017) {
```

Membandingkan elemen yang berjarak gap dengan elemen yang sedang diproses. Jika elemen sebelumnya lebih besar dari nilai sementara, maka elemen tersebut akan digeser ke kanan. Proses ini mirip dengan mekanisme Insertion Sort tetapi menggunakan jarak tertentu.

```
A_1017[j_1017] = A_1017[j_1017 - gap_1017];
j_1017 = j_1017 - gap_1017;
```

Menggeser elemen yang lebih besar ke posisi berikutnya. Indeks j_1017 kemudian dipindahkan mundur sesuai nilai gap. Proses ini berlangsung hingga ditemukan posisi yang tepat untuk elemen sementara.

```
A_1017[j_1017] = temp_1017;
```

Menempatkan nilai sementara ke posisi yang sesuai setelah proses pergeseran selesai.

```
gap_1017 = gap_1017/2;
```

Memperkecil nilai gap setelah satu tahap pengurutan selesai.

```
public static void main(String[] args) {
    // TODO Auto-generated method stub
    int[] data_1017 = {3,10,4,6,8,9,7,2,1,5};
```

method utama yang dijalankan pertama kali saat program dieksekusi. Array data_1017 dibuat sebagai data yang akan diurutkan menggunakan algoritma Shell Sort. Data tersebut berisi sepuluh bilangan yang belum terurut.

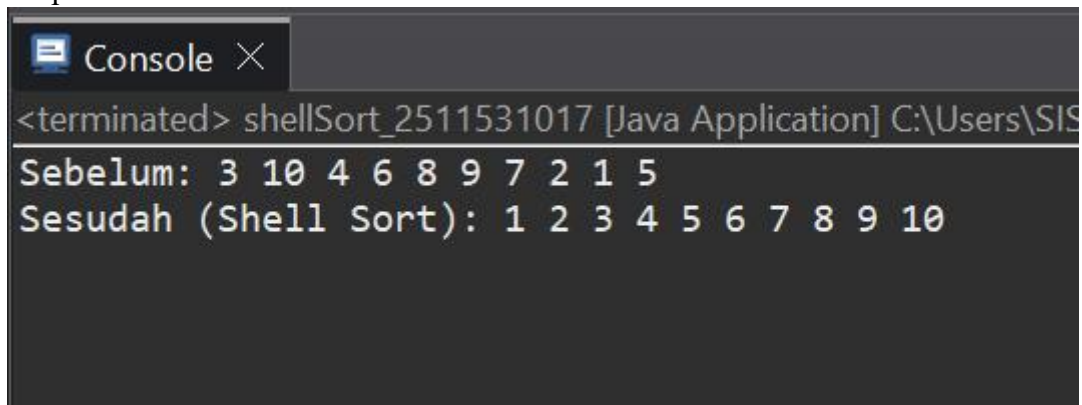
```
System.out.print("Sebelum: ");
printArray_1017(data_1017);
shellShort_2511531017(data_1017);
System.out.print("Sesudah (Shell Sort): ");
printArray_1017(data_1017);
```

menampilkan isi array sebelum dan sesudah proses sorting dilakukan. Method shellShort_2511531017() dipanggil untuk mengurutkan data. Hasil pengurutan kemudian ditampilkan ke layar.

```
public static void printArray_1017(int[] arr_1017) {
    for (int i_1017 : arr_1017) System.out.print(i_1017 + " ");
    System.out.println();
```

Menampilkan seluruh elemen array ke layar. Perulangan for-each digunakan agar setiap elemen array dapat dicetak secara berurutan.

Output:



```
<terminated> shellSort_2511531017 [Java Application] C:\Users\SIS
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10
```

Gambar 2.1

2.2 QuickSort_2511531017

```
package pekan8_2511531017;

public class QuickSort_2511531017 {
    static void swap_1017 (int[] arr_1017, int i_1017, int j_1017) {
        int temp_1017 = arr_1017 [i_1017];
        arr_1017[i_1017] = arr_1017[j_1017];
        arr_1017[j_1017] = temp_1017;
    }
    static void medianOfThree(int[] arr_1017, int low_1017, int high_1017)
    {
        int mid_1017 = low_1017 + (high_1017 - low_1017)/2;

        if (arr_1017[low_1017] > arr_1017 [mid_1017]) {
            swap_1017(arr_1017, low_1017, mid_1017);
        }
        if (arr_1017 [low_1017] > arr_1017 [high_1017]) {
            swap_1017(arr_1017, low_1017, high_1017);
        }
        if (arr_1017 [mid_1017] > arr_1017 [high_1017]) {
            swap_1017(arr_1017, low_1017, high_1017);
        }
        swap_1017(arr_1017, mid_1017, high_1017);
    }
    static int partition_1017 (int[] arr_1017, int low_1017, int high_1017)
    {
        medianOfThree(arr_1017, low_1017, high_1017);

        int pivot_1017 = arr_1017[high_1017];
        int i_1017 = (low_1017 - 1);

        for (int j_1017 = low_1017; j_1017 <= high_1017 - 1; j_1017++) {
            if (arr_1017[j_1017]<pivot_1017) {
                i_1017++;
                swap_1017(arr_1017,i_1017,j_1017);
            }
        }
    }
}
```

```

    }
    swap_1017 (arr_1017, i_1017 + 1, high_1017);
    return (i_1017 + 1);
}
static void quickSort_1017(int[] arr_1017, int low_1017, int high_1017)
{
    if (low_1017 < high_1017) {
        int pi_1017 = partition_1017(arr_1017, low_1017,
high_1017);

        quickSort_1017(arr_1017, low_1017, pi_1017 - 1);
        quickSort_1017(arr_1017, pi_1017 + 1, high_1017);
    }
}
public static void printArr_1017(int [] arr_1017) {
    for (int i_1017 = 0; i_1017 < arr_1017.length; i_1017++) {
        System.out.print(arr_1017[i_1017] + " ");
    }
    System.out.println();
}

public static void main(String[] args) {
    // TODO Auto-generated method stub
    int[] arr_1017 = {10,7,8,9,1,5};
    int N_1017 = arr_1017.length;
    System.out.print("Data sebelum diurutkan: ");
    printArr_1017(arr_1017);

    quickSort_1017(arr_1017, 0 , N_1017 - 1);

    System.out.print("Data Terurut quickSort: ");
    printArr_1017(arr_1017);

}
}

```

Kode 2.2

Penjelasan:

```

static void swap_1017 (int[] arr_1017, int i_1017, int j_1017) {
    int temp_1017 = arr_1017 [i_1017];
    arr_1017[i_1017] = arr_1017[j_1017];
    arr_1017[j_1017] = temp_1017;
}

```

Menukar posisi dua elemen dalam array. Nilai sementara disimpan terlebih dahulu pada variabel temp_1017 agar data tidak hilang saat pertukaran dilakukan. Method ini sering dipanggil selama proses Quick Sort berlangsung.

```

static void medianOfThree(int[] arr_1017, int low_1017, int high_1017) {
    int mid_1017 = low_1017 + (high_1017 - low_1017)/2;
}

```

Mencari indeks tengah dari array yang sedang diproses. Nilai tengah akan digunakan dalam proses pemilihan pivot.

```

if (arr_1017[low_1017] > arr_1017 [mid_1017]) {
    swap_1017(arr_1017, low_1017, mid_1017);
}

```

Jika elemen awal lebih besar, maka kedua elemen akan ditukar posisinya.

```

if (arr_1017 [low_1017] > arr_1017 [high_1017]) {
    swap_1017(arr_1017, low_1017, high_1017);
}
if (arr_1017 [mid_1017] > arr_1017 [high_1017]) {
    swap_1017(arr_1017, low_1017, high_1017);
}

```

Jika ditemukan urutan yang tidak sesuai maka dilakukan pertukaran posisi elemen.

```

swap_1017(arr_1017, mid_1017, high_1017);

```

Memindahkan nilai median ke posisi akhir array. Nilai tersebut nantinya akan digunakan sebagai pivot pada proses partition.

```

static int partition_1017 (int[] arr_1017, int low_1017, int high_1017) {
    medianOfThree(arr_1017, low_1017, high_1017);

    int pivot_1017 = arr_1017[high_1017];
}

```

Menentukan pivot terlebih dahulu menggunakan metode Median of Three. Setelah itu pivot diambil dari elemen terakhir array.

```

int i_1017 = (low_1017 - 1);

```

Menyimpan indeks elemen yang lebih kecil dari pivot. Nilai awal dibuat satu posisi sebelum indeks pertama array yang diproses.

```

for (int j_1017 = low_1017; j_1017 <= high_1017 - 1; j_1017++) {
}

```

Perulangan pada seluruh elemen array selain pivot. Setiap elemen akan dibandingkan dengan nilai pivot. Hasil perbandingan menentukan posisi elemen tersebut.

```

if (arr_1017[j_1017]<pivot_1017) {
    i_1017++;
    swap_1017(arr_1017,i_1017,j_1017);
}

```

Memindahkan elemen yang lebih kecil dari pivot ke bagian kiri array. Jika kondisi terpenuhi maka indeks i_1017 bertambah dan elemen ditukar posisinya.

```

swap_1017 (arr_1017, i_1017 + 1, high_1017);
return (i_1017 + 1);

```

Menempatkan pivot pada posisi yang benar setelah proses partition selesai. Nilai yang dikembalikan adalah indeks pivot yang baru.

```

static void quickSort_1017(int[] arr_1017, int low_1017, int high_1017) {
    if (low_1017 < high_1017) {
        int pi_1017 = partition_1017(arr_1017, low_1017,
high_1017);

        quickSort_1017(arr_1017, low_1017, pi_1017 - 1);
        quickSort_1017(arr_1017, pi_1017 + 1, high_1017);
    }
}

```

Jika jumlah elemen masih lebih dari satu maka proses sorting dilanjutkan. Jika tidak, proses akan berhenti secara otomatis. Method menjalankan proses partition dan memanggil quick sort secara rekursif. Bagian kiri dan kanan pivot akan diurutkan secara terpisah. Rekursi akan berakhir ketika seluruh elemen sudah berada pada posisi yang benar

```

public static void printArr_1017(int [] arr_1017) {
    for (int i_1017 = 0; i_1017 < arr_1017.length; i_1017++) {
        System.out.print(arr_1017[i_1017] + " ");
    }
}

```

```

    }
    System.out.println();

```

Menampilkan isi array ke layar. Seluruh elemen array dicetak secara berurutan menggunakan perulangan.

```

public static void main(String[] args) {
    // TODO Auto-generated method stub
    int[] arr_1017 = {10,7,8,9,1,5};
    int N_1017 = arr_1017.length;
    System.out.print("Data sebelum diurutkan: ");
    printArr_1017(arr_1017);

```

Membuat data awal dan menampilkan isi array sebelum proses pengurutan. Data tersebut masih berada dalam kondisi acak. Output awal membantu melihat perubahan setelah sorting dilakukan.

```

quickSort_1017(arr_1017, 0 , N_1017 - 1);

```

Menjalankan algoritma quick sort pada seluruh elemen array. Parameter pertama adalah array yang akan diurutkan. Parameter kedua dan ketiga menunjukkan indeks awal dan akhir array.

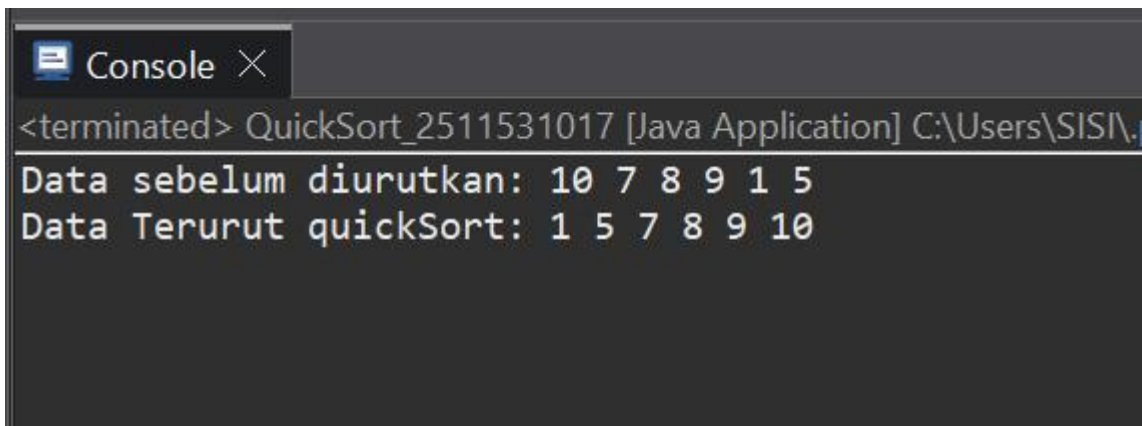
```

System.out.print("Data Terurut quickSort: ");
    printArr_1017(arr_1017);

```

Menampilkan hasil array setelah proses quick sort selesai. Seluruh data akan ditampilkan dalam keadaan terurut.

Output:



```

<terminated> QuickSort_2511531017 [Java Application] C:\Users\SISI\
Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quickSort: 1 5 7 8 9 10

```

Gambar 2.2

2.3 MergeSort_2511531017

```

package pekan8_2511531017;

public class MergeSort_2511531017 {
    void merge_1017(int arr_1017[], int l_1017, int m_1017, int r_1017) {
        int n1_1017 = m_1017 - l_1017 + 1;
        int n2_1017 = r_1017 - m_1017;

        int L_1017[] = new int[n1_1017];
        int R_1017[] = new int[n2_1017];

```

```

        for (int i_1017 = 0; i_1017 < n1_1017; ++i_1017)
            L_1017[i_1017] = arr_1017[l_1017 + i_1017];

        for (int j_1017 = 0; j_1017 < n2_1017; ++j_1017)
            R_1017[j_1017] = arr_1017[m_1017 + 1 + j_1017];

        int i_1017 = 0, j_1017 = 0;

        int k_1017 = l_1017;
        while (i_1017 < n1_1017 && j_1017 < n2_1017) {
            if (L_1017[i_1017] <= R_1017[j_1017]) {
                arr_1017[k_1017] = L_1017[i_1017];
                i_1017++;
            } else {
                arr_1017[k_1017] = R_1017[j_1017];
                j_1017++;
            }
            k_1017++;
        }

        while (i_1017 < n1_1017) {
            arr_1017[k_1017] = L_1017[i_1017];
            i_1017++;
            k_1017++;
        }

        while (j_1017 < n2_1017) {
            arr_1017[k_1017] = R_1017[j_1017];
            j_1017++;
            k_1017++;
        }
    }

    void sort_1017(int arr_1017[], int l_1017, int r_1017) {
        if (l_1017 < r_1017) {
            int m_1017 = (l_1017 + r_1017) / 2;

            sort_1017(arr_1017, l_1017, m_1017);
            sort_1017(arr_1017, m_1017 + 1, r_1017);

            merge_1017(arr_1017, l_1017, m_1017, r_1017);
        }
    }

    static void printArray_1017(int arr_1017[]) {
        int n_1017 = arr_1017.length;

        for (int i_1017 = 0; i_1017 < n_1017; ++i_1017)
            System.out.print(arr_1017[i_1017] + " ");

        System.out.println();
    }

    public static void main(String[] args) {

        int arr_1017[] = {12, 11, 13, 5, 6, 7};

```

```

        System.out.println("Sebelum terurut");
        printArray_1017(arr_1017);

        MergeSort_2511531017 ob_1017 = new MergeSort_2511531017();

        ob_1017.sort_1017(arr_1017, 0, arr_1017.length - 1);

        System.out.println("\nSesudah Terurut menggunakan Merge Sort");
        printArray_1017(arr_1017);
    }
}

```

Kode 2.3

Penjelasan:

```

void merge_1017(int arr_1017[], int l_1017, int m_1017, int r_1017) {
    int n1_1017 = m_1017 - l_1017 + 1;
    int n2_1017 = r_1017 - m_1017;

    int L_1017[] = new int[n1_1017];
    int R_1017[] = new int[n2_1017];

    for (int i_1017 = 0; i_1017 < n1_1017; ++i_1017)
        L_1017[i_1017] = arr_1017[l_1017 + i_1017];

    for (int j_1017 = 0; j_1017 < n2_1017; ++j_1017)
        R_1017[j_1017] = arr_1017[m_1017 + 1 + j_1017];
}

```

Untuk menentukan ukuran sub-array kiri dan kanan, kemudian membuat array sementara L_1017 dan R_1017. Setelah itu data dari array utama disalin ke kedua array sementara tersebut. Tujuannya agar proses penggabungan data dapat dilakukan dengan lebih mudah.

```

int i_1017 = 0, j_1017 = 0;

int k_1017 = l_1017;
while (i_1017 < n1_1017 && j_1017 < n2_1017) {
    if (L_1017[i_1017] <= R_1017[j_1017]) {
        arr_1017[k_1017] = L_1017[i_1017];
        i_1017++;
    } else {
        arr_1017[k_1017] = R_1017[j_1017];
        j_1017++;
    }
    k_1017++;
}

```

Membandingkan elemen pada array kiri dan array kanan. Nilai yang lebih kecil akan dimasukkan terlebih dahulu ke array utama sehingga urutan data tetap terjaga. Proses ini berlangsung sampai salah satu array habis dibandingkan.

```

while (i_1017 < n1_1017) {
    arr_1017[k_1017] = L_1017[i_1017];
    i_1017++;
    k_1017++;
}

while (j_1017 < n2_1017) {
    arr_1017[k_1017] = R_1017[j_1017];
}

```

```
j_1017++;  
k_1017++;
```

Menyalin sisa elemen yang masih terdapat pada salah satu array sementara. Kondisi ini terjadi jika array kiri atau array kanan masih memiliki data setelah proses perbandingan selesai. Seluruh sisa elemen tersebut langsung dipindahkan ke array utama agar proses penggabungan selesai dengan sempurna.

```
void sort_1017(int arr_1017[], int l_1017, int r_1017) {  
    if (l_1017 < r_1017) {  
        int m_1017 = (l_1017 + r_1017) / 2;  
  
        sort_1017(arr_1017, l_1017, m_1017);  
        sort_1017(arr_1017, m_1017 + 1, r_1017);  
  
        merge_1017(arr_1017, l_1017, m_1017, r_1017);  
    }  
}
```

Menggunakan teknik rekursif, array akan terus dibagi menjadi dua bagian menggunakan indeks tengah sampai setiap bagian hanya berisi satu elemen. Setelah proses pembagian selesai, method `merge_1017()` dipanggil untuk menggabungkan kembali bagian-bagian tersebut hingga seluruh data menjadi terurut.

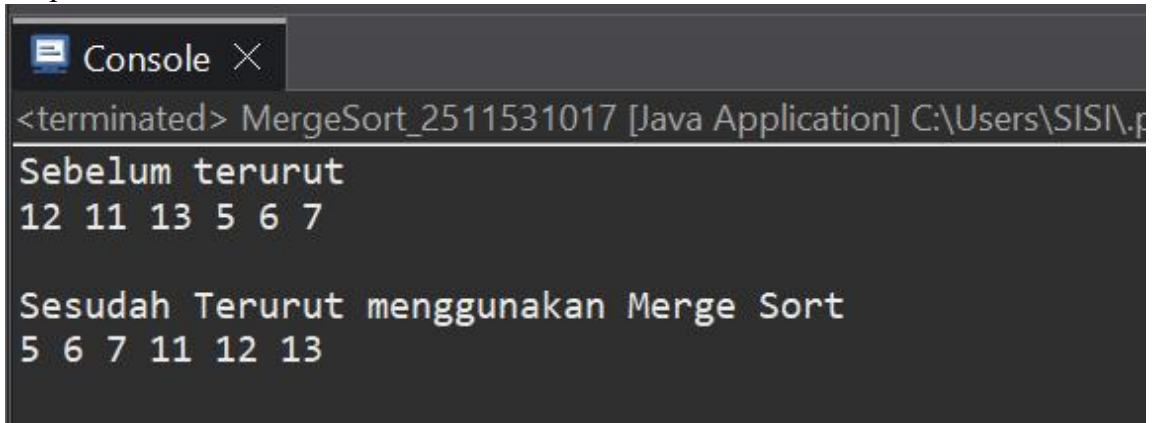
```
static void printArray_1017(int arr_1017[]) {  
    int n_1017 = arr_1017.length;  
  
    for (int i_1017 = 0; i_1017 < n_1017; ++i_1017)  
        System.out.print(arr_1017[i_1017] + " ");  
  
    System.out.println();  
}
```

Menampilkan seluruh isi array ke layar. Program melakukan perulangan dari indeks pertama hingga indeks terakhir dan mencetak setiap elemen array secara berurutan. Method ini digunakan untuk menampilkan data sebelum dan sesudah proses sorting dilakukan.

```
public static void main(String[] args) {  
  
    int arr_1017[] = {12, 11, 13, 5, 6, 7};  
  
    System.out.println("Sebelum terurut");  
    printArray_1017(arr_1017);  
  
    MergeSort_2511531017 ob_1017 = new MergeSort_2511531017();  
  
    ob_1017.sort_1017(arr_1017, 0, arr_1017.length - 1);  
  
    System.out.println("\nSesudah Terurut menggunakan Merge Sort");  
    printArray_1017(arr_1017);  
}
```

Method utama yang pertama kali dijalankan saat program dieksekusi. Pada method ini dibuat data array yang akan diurutkan, kemudian program menampilkan data awal dan menjalankan algoritma Merge Sort melalui method `sort_1017()`. Setelah proses pengurutan selesai, hasil akhir array yang sudah terurut ditampilkan ke layar.

Output:



```
<terminated> MergeSort_2511531017 [Java Application] C:\Users\SISI\p
Sebelum terurut
12 11 13 5 6 7

Sesudah Terurut menggunakan Merge Sort
5 6 7 11 12 13
```

Gambar 2.3

2.4 BubleGUI_2511531017

```
package pekan8_2511531017;

import javax.swing.*;

public class BubleGUI_2511531017 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int[] array_1017;
    private JLabel[] labelArray_1017;
    JButton stepButton_1017;
    private JButton resetButton_1017;
    JButton setButton_1017;
    private JTextField inputField_1017;
    private JPanel panelArray_1017;
    private JTextArea stepArea_1017;

    private int i_1017 = 1, j_1017;
    private boolean sorting_1017 = false;
    private int stepCount_1017 = 1;

    /**
     * Create the frame.
     */
    public BubleGUI_2511531017() {
        setTitle("Insertion Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        JPanel inputPanel_1017 = new JPanel(new FlowLayout());
        inputField_1017 = new JTextField(30);
        setButton_1017 = new JButton("Set Array");
        inputPanel_1017.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
```

```

inputPanel_1017.add(inputField_1017);
inputPanel_1017.add(setButton_1017);

panelArray_1017 = new JPanel();
panelArray_1017.setLayout(new FlowLayout());

JPanel controlPanel_1017 = new JPanel();
stepButton_1017 = new JButton ("Langkah Selanjutnya ");
resetButton_1017 = new JButton ("Reset");
stepButton_1017.setEnabled(false);
controlPanel_1017.add(stepButton_1017);
controlPanel_1017.add(resetButton_1017);

stepArea_1017 = new JTextArea(8, 60);
stepArea_1017.setEditable(false);
stepArea_1017.setFont(new Font("Monospaced", Font.PLAIN, 14));
JScrollPane scrollPane = new JScrollPane(stepArea_1017);

add(inputPanel_1017, BorderLayout.NORTH);
add(panelArray_1017, BorderLayout.CENTER);
add(controlPanel_1017, BorderLayout.SOUTH);
add(scrollPane, BorderLayout.EAST);

setButton_1017.addActionListener(e -> setArrayFromInput_1017());
stepButton_1017.addActionListener(e -> performStep_1017());
resetButton_1017.addActionListener(e -> reset_1017());
}

private void setArrayFromInput_1017() {
    String text_1017 = inputField_1017.getText().trim();

    if (text_1017.isEmpty()) {
        return;
    }

    String[] parts_1017 = text_1017.split(",");
    array_1017 = new int[parts_1017.length];

    try {
        for (int k = 0; k < parts_1017.length; k++) {
            array_1017[k] = Integer.parseInt(parts_1017[k].trim());
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(
            this,
            "Masukkan hanya angka yang dipisahkan koma!",
            "Error",
            JOptionPane.ERROR_MESSAGE
        );
        return;
    }

    i_1017 = 0;
    j_1017 = 0;
    stepCount_1017 = 1;
    sorting_1017 = true;

    stepButton_1017.setEnabled(true);
    stepArea_1017.setText("");
    panelArray_1017.removeAll();

    labelArray_1017 = new JLabel[array_1017.length];

    for (int k = 0; k < array_1017.length; k++) {
        labelArray_1017[k] = new JLabel(String.valueOf(array_1017[k]));
        labelArray_1017[k].setFont(new Font("Arial", Font.BOLD, 24));
        labelArray_1017[k].setOpaque(true);
    }
}

```

```

        labelArray_1017[k].setBackground(Color.WHITE);
        labelArray_1017[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
        labelArray_1017[k].setPreferredSize(new Dimension(50, 50));
        labelArray_1017[k].setHorizontalAlignment(SwingConstants.CENTER);

        panelArray_1017.add(labelArray_1017[k]);
    }

    panelArray_1017.revalidate();
    panelArray_1017.repaint();
}

private void performStep_1017() {
    if (!sorting_1017 || i_1017 >= array_1017.length - 1) {
        sorting_1017 = false;
        stepButton_1017.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
        return;
    }

    resetHighlights_1017();

    StringBuilder stepLog_1017 = new StringBuilder();

    labelArray_1017[j_1017].setBackground(Color.CYAN);
    labelArray_1017[j_1017 + 1].setBackground(Color.CYAN);

    if (array_1017[j_1017] > array_1017[j_1017 + 1]) {
        // Swap
        int temp = array_1017[j_1017];
        array_1017[j_1017] = array_1017[j_1017 + 1];
        array_1017[j_1017 + 1] = temp;

        labelArray_1017[j_1017].setBackground(Color.RED);
        labelArray_1017[j_1017 + 1].setBackground(Color.RED);

        stepLog_1017.append("Langkah ").append(stepCount_1017).append(": ")
            .append("Menukar elemen ke-")
            .append(j_1017)
            .append(" (").append(array_1017[j_1017 + 1]).append(")")
            .append(" dengan ke-")
            .append(j_1017 + 1)
            .append(" (").append(array_1017[j_1017]).append(")\n");
    } else {
        stepLog_1017.append("Langkah ").append(stepCount_1017).append(": ")
            .append("Tidak ada pertukaran antara ke-")
            .append(j_1017)
            .append(" dan ke-")
            .append(j_1017 + 1)
            .append("\n");
    }

    stepLog_1017.append("Hasil: ").append(arrayToString_1017(array_1017)).append("\n\n");
    stepArea_1017.append(stepLog_1017.toString());

    updateLabels_1017();

    j_1017++;

    if (j_1017 >= array_1017.length - 1 - i_1017) {
        j_1017 = 0;
        i_1017++;
    }

    stepCount_1017++;

    if (i_1017 >= array_1017.length - 1) {
        sorting_1017 = false;
        stepButton_1017.setEnabled(false);
        resetHighlights_1017();
    }
}

```

```

        for (JLabel label : labelArray_1017) {
            label.setBackground(Color.GREEN);
        }

        JOptionPane.showMessageDialog(this, "Sorting selesai!");
    }

    private void updateLabels_1017() {
        for (int k_1017 = 0; k_1017 < array_1017.length; k_1017++) {
            labelArray_1017[k_1017].setText(String.valueOf(array_1017[k_1017]));
        }
    }

    private void resetHighlights_1017() {
        for (JLabel label : labelArray_1017) {
            label.setBackground(Color.WHITE);
        }
    }

    private void reset_1017() {
        inputField_1017.setText("");
        panelArray_1017.removeAll();
        panelArray_1017.revalidate();
        panelArray_1017.repaint();

        stepArea_1017.setText("");
        stepButton_1017.setEnabled(false);

        sorting_1017 = false;
        i_1017 = 0;
        j_1017 = 0;
        stepCount_1017 = 1;
    }

    private String arrayToString_1017(int[] arr_1017) {
        StringBuilder sb_1017 = new StringBuilder();

        for (int k_1017 = 0; k_1017 < arr_1017.length; k_1017++) {
            sb_1017.append(arr_1017[k_1017]);

            if (k_1017 < arr_1017.length - 1) {
                sb_1017.append(", ");
            }
        }

        return sb_1017.toString();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                new BubleGUI_2511531017().setVisible(true);
            }
        });
    }
}

```

Kode 2.4

Penjelasan:

```

public class BubleGUI_2511531017 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int[] array_1017;
    private JLabel[] labelArray_1017;
    private JButton stepButton_1017;
    private JButton resetButton_1017;
    private JButton setButton_1017;
    private JTextField inputField_1017;
    private JPanel panelArray_1017;
    private JTextArea stepArea_1017;

```

Mendeklarasikan variabel dan komponen GUI yang dibutuhkan program.

```

    private int i_1017 = 1, j_1017;
    private boolean sorting_1017 = false;

```

```
private int stepCount_1017 = 1;
```

Variabel `i_1017` dan `j_1017` digunakan sebagai indeks sorting, sedangkan `stepCount_1017` digunakan untuk menghitung jumlah langkah sorting.

```
public BubleGUI_2511531017() {  
    setTitle("Buble Sort Langkah per Langkah");  
    setSize(750,400);  
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    setLocationRelativeTo(null);  
    setLayout(new BorderLayout());  
}
```

Konstruktor yang digunakan untuk mengatur tampilan utama GUI. Program mengatur judul window, ukuran frame, posisi tampilan, dan layout yang digunakan.

```
JPanel inputPanel_1017 = new JPanel(new FlowLayout());  
    inputField_1017 = new JTextField(30);  
    setButton_1017 = new JButton("Set Array");  
    inputPanel_1017.add(new JLabel("Masukkan angka (pisahkan dengan  
koma):"));
```

Kode ini digunakan untuk membuat bagian input array pada GUI. Pengguna dapat memasukkan angka ke dalam text field dengan format dipisahkan tanda koma.

```
panelArray_1017 = new JPanel();  
panelArray_1017.setLayout(new FlowLayout());
```

Membuat panel yang berfungsi menampilkan isi array pada GUI. Data array nantinya akan ditampilkan dalam bentuk label angka.

```
JPanel controlPanel_3023 = new JPanel();  
    stepButton_1017 = new JButton ("Langkah Selanjutnya ");  
    resetButton_1017 = new JButton ("Reset");  
    stepButton_1017.setEnabled(false);
```

Digunakan untuk membuat tombol kontrol program. Tombol langkah digunakan untuk menjalankan sorting satu tahap, sedangkan tombol reset digunakan untuk mengembalikan program ke kondisi awal.

```
stepArea_1017 = new JTextArea(8, 60);  
stepArea_1017.setEditable(false);
```

Membuat area teks yang menampilkan langkah-langkah sorting. Area teks dibuat tidak bisa diedit oleh pengguna.

```
stepArea_1017.setFont(new Font("Monospaced", Font.PLAIN, 14));  
JScrollPane scrollPane = new JScrollPane(stepArea_1017);
```

Mengatur font pada area langkah sorting dan menambahkan scroll pane agar teks dapat discroll.

```
add(inputPanel_1017, BorderLayout.NORTH);  
add(panelArray_1017, BorderLayout.CENTER);  
add(controlPanel_3023, BorderLayout.SOUTH);  
add(scrollPane, BorderLayout.EAST);
```

Menempatkan semua komponen GUI pada posisi layout masing-masing.

```
setButton_1017.addActionListener(e -> setArrayFromInput_1017());  
stepButton_1017.addActionListener(e -> performStep_1017());  
resetButton_1017.addActionListener(e -> reset_1017());
```

Memberikan aksi pada setiap tombol GUI.

```
private void setArrayFromInput_1017() {
```

```
String text_1017 = inputField_1017.getText().trim();
if(text_1017.isEmpty()) return;
String[] parts_1017 = text_1017.split(",");
array_1017 = new int [parts_1017.length];
```

Untuk mengambil input angka dari pengguna lalu memisahkannya berdasarkan tanda koma. Setelah itu program membuat array integer sesuai jumlah data yang dimasukkan.

```
try {
    for (int k = 0; k < parts_1017.length; k++) {
        array_1017[k] =
Integer.parseInt(parts_1017[k].trim());}
    }catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Masukan hanya
angka dipisahkan "+ "dengan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
        return; }
```

mengubah input string menjadi integer dan menangani kesalahan input. Jika pengguna memasukkan selain angka, program akan menampilkan pesan error menggunakan JOptionPane.

```
i_1017 = 0;
j_1017 = 0;
stepCount_1017 = 1;
sorting_1017 = true;
stepButton_1017.setEnabled(true);
stepArea_1017.setText("");
panelArray_1017.removeAll();
```

menginisialisasi ulang variabel yang mengontrol proses Bubble Sort sebelum sorting dimulai. Variabel i_1017 dan j_1017 diatur ke nilai awal sebagai penunjuk indeks perulangan, sedangkan stepCount_1017 diatur menjadi 1 untuk menghitung jumlah langkah yang dilakukan. Variabel sorting_1017 bernilai true untuk menandakan bahwa proses pengurutan sedang aktif dan siap dijalankan.

```
labelArray_1017 = new JLabel [array_1017.length];
for (int k = 0; k < array_1017.length; k++) {
    labelArray_1017[k] = new
JLabel(String.valueOf(array_1017[k]));
    labelArray_1017[k].setFont(new Font("Arial",Font.BOLD,24));

    labelArray_1017[k].setBorder(BorderFactory.createLineBorder(Color.BLACK
));
    labelArray_1017[k].setPreferredSize(new Dimension (50,50));

    labelArray_1017[k].setHorizontalAlignment(SwingConstants.CENTER);
    panelArray_1017.add(labelArray_1017[k]);
```

Digunakan untuk membuat visualisasi array pada GUI menggunakan label. Setiap elemen array ditampilkan dalam bentuk kotak angka dengan ukuran, border, dan posisi teks yang sudah diatur.

```
panelArray_1017.revalidate();
panelArray_1017.repaint();
```

Memperbarui tampilan panel setelah array berhasil ditampilkan ke GUI.

```

private void performStep_1017() {
    if (!sorting_1017 || i_1017 >= array_1017.length - 1) {
        sorting_1017 = false;
        stepButton_1017.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
        return;
    }

    resetHighlights_1017();

    StringBuilder stepLog_1017 = new StringBuilder();

    labelArray_1017[j_1017].setBackground(Color.CYAN);
    labelArray_1017[j_1017 + 1].setBackground(Color.CYAN);

```

Memeriksa apakah proses sorting masih dapat dilanjutkan atau sudah selesai. Jika sorting selesai, tombol langkah berikutnya akan dinonaktifkan dan muncul pesan bahwa proses pengurutan telah selesai. Selain itu, program juga menghapus warna sebelumnya dan memberi warna biru pada dua elemen yang sedang dibandingkan.

```

if (array_1017[j_1017] > array_1017[j_1017 + 1]) {
    // Swap
    int temp = array_1017[j_1017];
    array_1017[j_1017] = array_1017[j_1017 + 1];
    array_1017[j_1017 + 1] = temp;

    labelArray_1017[j_1017].setBackground(Color.RED);
    labelArray_1017[j_1017 + 1].setBackground(Color.RED);

    stepLog_1017.append("Langkah ").append(stepCount_1017).append(":
")
                .append("Menukar elemen ke-").append(j_1017)
                .append(" ").append(array_1017[j_1017 + 1]).append(")")
                .append(" dengan ke-").append(j_1017 + 1)
                .append(" ").append(array_1017[j_1017]).append("\n");

```

Menjalankan proses pertukaran data apabila elemen kiri lebih besar daripada elemen kanan. Pertukaran dilakukan menggunakan variabel sementara temp agar nilai tidak hilang saat dipindahkan. Setelah pertukaran berhasil dilakukan, kedua label diberi warna merah dan informasi langkah ditambahkan ke area log.

```

else {
    stepLog_1017.append("Langkah ").append(stepCount_1017).append(":
")
                .append("Tidak ada pertukaran antara ke-")
                .append(j_1017).append(" dan ke-")
                .append(j_1017 + 1).append("\n");
}

stepLog_1017.append("Hasil:
").append(arrayToString_1017(array_1017)).append("\n\n");
stepArea_1017.append(stepLog_1017.toString());

updateLabels_1017();

```

Dijalankan ketika tidak terjadi pertukaran data karena urutan elemen sudah benar. Program tetap mencatat hasil perbandingan dan menampilkan kondisi array saat itu ke area log. Setelah itu tampilan label diperbarui agar sesuai dengan isi array terbaru.

```
j_1017++;

    if (j_1017 >= array_1017.length - 1 - i_1017) {
        j_1017 = 0;
        i_1017++;
    }

    stepCount_1017++;

    if (i_1017 >= array_1017.length - 1) {
        sorting_1017 = false;
        stepButton_1017.setEnabled(false);
        resetHighlights_1017();

        for (JLabel label : labelArray_1017) {
            label.setBackground(Color.GREEN);
        }

        JOptionPane.showMessageDialog(this, "Sorting selesai!");
    }
```

Mengatur perpindahan indeks Bubble Sort setelah satu perbandingan selesai dilakukan. Ketika satu putaran telah selesai, nilai i_1017 akan bertambah dan proses akan dilanjutkan ke putaran berikutnya. Jika seluruh data sudah terurut, semua label akan diberi warna hijau, tombol langkah dinonaktifkan, dan program menampilkan pesan bahwa sorting telah selesai.

```
private void updateLabels_1017() {
    for (int k_1017 = 0; k_1017 < array_1017.length; k_1017++) {
        labelArray_1017[k_1017].setText(String.valueOf(array_1017[k_1017]));
    }
}
```

Memperbarui isi label array pada GUI agar sesuai dengan hasil sorting terbaru.

```
private void resetHighlights_1017() {
    for (JLabel label : labelArray_1017) {
        label.setBackground(Color.WHITE);
    }
}
```

Mengembalikan warna seluruh label ke warna putih. Tujuannya agar penanda perbandingan atau pertukaran pada langkah sebelumnya tidak tetap terlihat. Method ini dipanggil setiap kali proses langkah baru dimulai.

```
private void reset_1017() {
    inputField_1017.setText("");
    panelArray_1017.removeAll();
    panelArray_1017.revalidate();
    panelArray_1017.repaint();

    stepArea_1017.setText("");
    stepButton_1017.setEnabled(false);

    sorting_1017 = false;
    i_1017 = 0;
    j_1017 = 0;
    stepCount_1017 = 1;
}
```

Mengembalikan program ke kondisi awal. Seluruh input, tampilan array, dan log proses sorting akan dihapus dari GUI. Selain itu seluruh variabel kontrol sorting juga dikembalikan ke nilai awal.

```
private String arrayToString_1017(int[] arr_1017) {
    StringBuilder sb_1017 = new StringBuilder();

    for (int k_1017 = 0; k_1017 < arr_1017.length; k_1017++) {
        sb_1017.append(arr_1017[k_1017]);

        if (k_1017 < arr_1017.length - 1) {
            sb_1017.append(", ");
        }
    }

    return sb_1017.toString();
}
```

Mengubah isi array menjadi string agar dapat ditampilkan pada area langkah sorting.

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
        @Override
        public void run() {
            new BubleGUI_2511531017().setVisible(true);
        }
    });
}
```

Membuat objek BubleGUI_2511531017 dan menampilkannya ke layar. Dengan demikian pengguna dapat menjalankan dan menggunakan aplikasi Bubble Sort GUI.

Output:

Buble Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

4	7	2
3	13	5
	12	

Buble Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

4	7	2
3	13	5
	12	

Langkah 1: Tidak ada pertukaran antara ke-0 dan ke-1
Hasil: 4, 7, 2, 3, 13, 5, 12

Buble Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

4	2	7
3	13	5
	12	

Langkah 1: Tidak ada pertukaran antara ke-0 dan ke-1
Hasil: 4, 7, 2, 3, 13, 5, 12

Langkah 2: Menukar elemen ke-1 (7) dengan ke-2 (2)
Hasil: 4, 2, 7, 3, 13, 5, 12

Buble Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

4	2	3
7	13	5
	12	

Langkah 1: Tidak ada pertukaran antara ke-0 dan ke-1
Hasil: 4, 7, 2, 3, 13, 5, 12

Langkah 2: Menukar elemen ke-1 (7) dengan ke-2 (2)
Hasil: 4, 2, 7, 3, 13, 5, 12

Langkah 3: Menukar elemen ke-2 (7) dengan ke-3 (3)
Hasil: 4, 2, 3, 7, 13, 5, 12

Buble Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

4	2	3
7	13	5
12		

Langkah 1: Tidak ada pertukaran antara ke-0 dan ke-1
Hasil: 4, 7, 2, 3, 13, 5, 12

Langkah 2: Menukar elemen ke-1 (7) dengan ke-2 (2)
Hasil: 4, 2, 7, 3, 13, 5, 12

Langkah 3: Menukar elemen ke-2 (7) dengan ke-3 (3)
Hasil: 4, 2, 3, 7, 13, 5, 12

Langkah 4: Tidak ada pertukaran antara ke-3 dan ke-4
Hasil: 4, 2, 3, 7, 13, 5, 12

Buble Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

4	2	3
7	5	13
12		

Langkah 2: Menukar elemen ke-1 (7) dengan ke-2 (2)
Hasil: 4, 2, 7, 3, 13, 5, 12

Langkah 3: Menukar elemen ke-2 (7) dengan ke-3 (3)
Hasil: 4, 2, 3, 7, 13, 5, 12

Langkah 4: Tidak ada pertukaran antara ke-3 dan ke-4
Hasil: 4, 2, 3, 7, 13, 5, 12

Langkah 5: Menukar elemen ke-4 (13) dengan ke-5 (5)
Hasil: 4, 2, 3, 7, 5, 13, 12

Buble Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma): 4,7,2,3,13,5,12

Set Array

4	2	3
7	5	12
	13	

Hasil: 4, 2, 7, 3, 13, 5, 12

Langkah 3: Menukar elemen ke-2 (7) dengan ke-3 (3)

Hasil: 4, 2, 3, 7, 13, 5, 12

Langkah 4: Tidak ada pertukaran antara ke-3 dan ke-4

Hasil: 4, 2, 3, 7, 13, 5, 12

Langkah 5: Menukar elemen ke-4 (13) dengan ke-5 (5)

Hasil: 4, 2, 3, 7, 5, 13, 12

Langkah 6: Menukar elemen ke-5 (13) dengan ke-6 (12)

Hasil: 4, 2, 3, 7, 5, 12, 13

Langkah Selanjutnya

Reset

Buble Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma): 4,7,2,3,13,5,12

Set Array

2	4	3
7	5	12
	13	

Hasil: 4, 2, 3, 7, 13, 5, 12

Langkah 4: Tidak ada pertukaran antara ke-3 dan ke-4

Hasil: 4, 2, 3, 7, 13, 5, 12

Langkah 5: Menukar elemen ke-4 (13) dengan ke-5 (5)

Hasil: 4, 2, 3, 7, 5, 13, 12

Langkah 6: Menukar elemen ke-5 (13) dengan ke-6 (12)

Hasil: 4, 2, 3, 7, 5, 12, 13

Langkah 7: Menukar elemen ke-0 (4) dengan ke-1 (2)

Hasil: 2, 4, 3, 7, 5, 12, 13

Langkah Selanjutnya

Reset

Buble Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

Set Array

2

3

4

7

5

12

13

Hasil: 4, 2, 3, 7, 5, 13, 12

Langkah 5: Menukar elemen ke-4 (13) dengan ke-5 (5)

Hasil: 4, 2, 3, 7, 5, 13, 12

Langkah 6: Menukar elemen ke-5 (13) dengan ke-6 (12)

Hasil: 4, 2, 3, 7, 5, 12, 13

Langkah 7: Menukar elemen ke-0 (4) dengan ke-1 (2)

Hasil: 2, 4, 3, 7, 5, 12, 13

Langkah 8: Menukar elemen ke-1 (4) dengan ke-2 (3)

Hasil: 2, 3, 4, 7, 5, 12, 13

Langkah Selanjutnya

Reset

Buble Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

Set Array

2

3

4

7

5

12

13

Hasil: 4, 2, 3, 7, 5, 13, 12

Langkah 6: Menukar elemen ke-5 (13) dengan ke-6 (12)

Hasil: 4, 2, 3, 7, 5, 12, 13

Langkah 7: Menukar elemen ke-0 (4) dengan ke-1 (2)

Hasil: 2, 4, 3, 7, 5, 12, 13

Langkah 8: Menukar elemen ke-1 (4) dengan ke-2 (3)

Hasil: 2, 3, 4, 7, 5, 12, 13

Langkah 9: Tidak ada pertukaran antara ke-2 dan ke-3

Hasil: 2, 3, 4, 7, 5, 12, 13

Langkah Selanjutnya

Reset

Buble Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

2	3	4
5	7	12
	13	

Langkah 7: Menukar elemen ke-0 (4) dengan ke-1 (2)
 Hasil: 2, 4, 3, 7, 5, 12, 13

Langkah 8: Menukar elemen ke-1 (4) dengan ke-2 (3)
 Hasil: 2, 3, 4, 7, 5, 12, 13

Langkah 9: Tidak ada pertukaran antara ke-2 dan ke-3
 Hasil: 2, 3, 4, 7, 5, 12, 13

Langkah 10: Menukar elemen ke-3 (7) dengan ke-4 (5)
 Hasil: 2, 3, 4, 5, 7, 12, 13

Buble Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

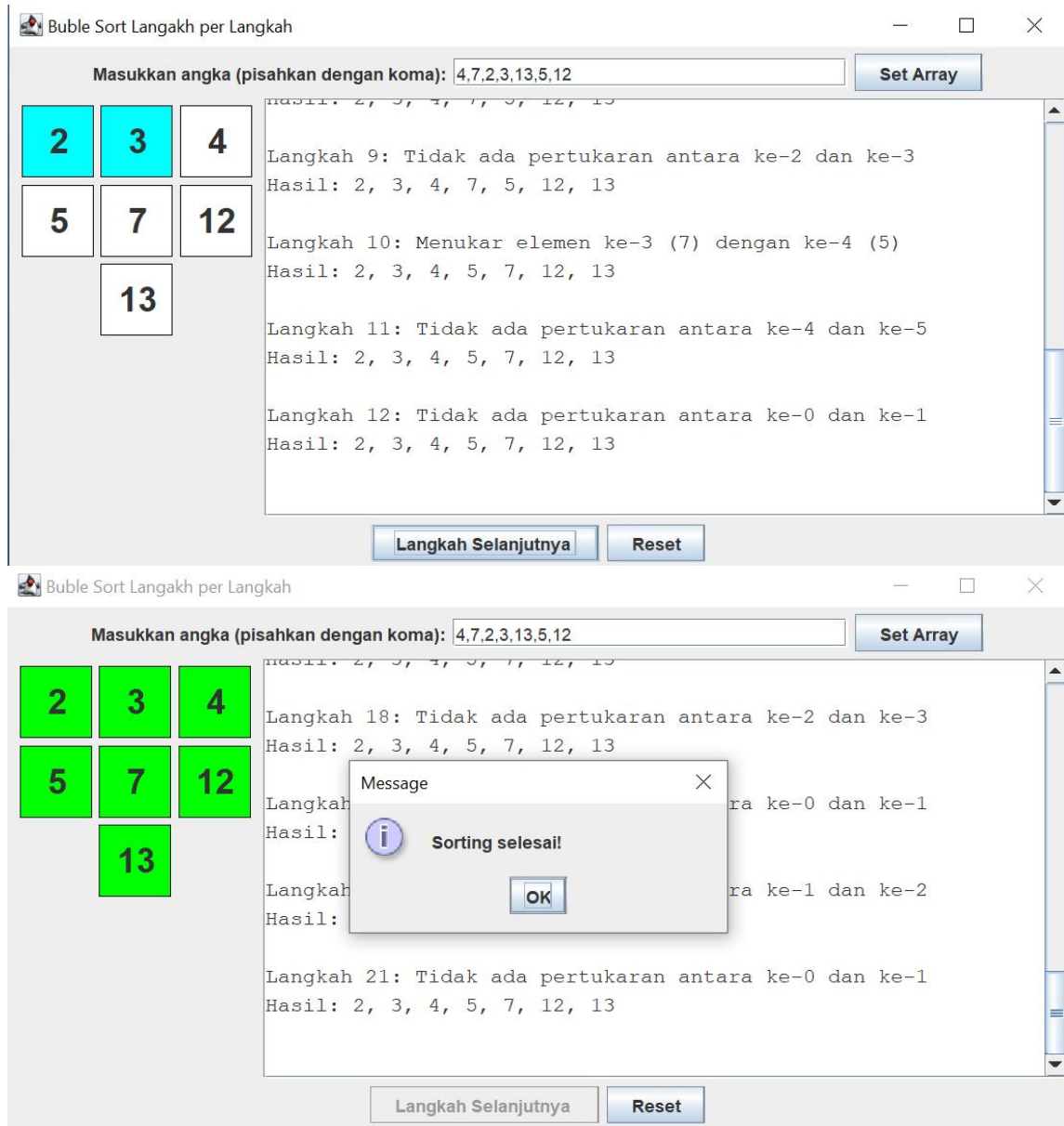
2	3	4
5	7	12
	13	

Langkah 8: Menukar elemen ke-1 (4) dengan ke-2 (3)
 Hasil: 2, 3, 4, 7, 5, 12, 13

Langkah 9: Tidak ada pertukaran antara ke-2 dan ke-3
 Hasil: 2, 3, 4, 7, 5, 12, 13

Langkah 10: Menukar elemen ke-3 (7) dengan ke-4 (5)
 Hasil: 2, 3, 4, 5, 7, 12, 13

Langkah 11: Tidak ada pertukaran antara ke-4 dan ke-5
 Hasil: 2, 3, 4, 5, 7, 12, 13



Gambar 2.4

TUGAS

Shellsort

Alasan:

Pemilihan Shell Sort karena memiliki proses pengurutan yang lebih sederhana dibandingkan Quick Sort dan Merge Sort. Algoritma ini menggunakan konsep gap untuk membandingkan elemen yang berjauhan sehingga dapat mempercepat proses pengurutan dibandingkan Insertion Sort biasa. Dibandingkan Merge Sort, Shell Sort tidak membutuhkan memori tambahan untuk menyimpan array sementara selama proses pengurutan. Selain itu, implementasi Shell Sort lebih mudah karena tidak menggunakan rekursi seperti Quick Sort maupun Merge Sort. Dengan kompleksitas dan implementasi yang relatif sederhana, Shell Sort menjadi pilihan yang sesuai untuk memahami konsep sorting pada praktikum ini.

2.5 Lagu_2511531017

```
package pekan8_2511531017;

public class Lagu_2511531017 {
    String judul_1017;
    String penyanyi_1017;
    int durasi_1017;
    public Lagu_2511531017(String judul_1017, String penyanyi_1017, int
durasi_1017) {
        this.judul_1017 = judul_1017;
        this.penyanyi_1017 = penyanyi_1017;
        this.durasi_1017 = durasi_1017;
    }
}
```

Kode 2.5

Penjelasan:

Class ini digunakan untuk membuat class yang merepresentasikan data lagu dalam playlist. Setiap objek lagu menyimpan informasi berupa judul lagu, nama penyanyi, dan durasi lagu. Class ini nantinya digunakan sebagai tipe data utama yang akan diurutkan menggunakan algoritma Shell Sort.

2.6 Sorting_2511531017

```
package pekan8_2511531017;

public class Sorting_2511531017 {

    static Lagu_2511531017[] dataLagu_1017 = new Lagu_2511531017[20];
    static int jumlahData_1017 = 0;
    public static void inputData_1017() {
        dataLagu_1017[0] = new Lagu_2511531017("Perfect", "Ed Sheeran", 263);
    }
}
```

```

211);
dataLagu_1017[1] = new Lagu_2511531017("Attention", "Charlie Puth",
204);
dataLagu_1017[2] = new Lagu_2511531017("Believer", "Imagine Dragons",
240);
dataLagu_1017[3] = new Lagu_2511531017("Shape Of You", "Ed Sheeran",
dataLagu_1017[4] = new Lagu_2511531017("Dandelions", "Ruth B", 233);
dataLagu_1017[5] = new Lagu_2511531017("Memories", "Maroon 5", 189);
dataLagu_1017[6] = new Lagu_2511531017("Photograph", "Ed Sheeran",
258);
dataLagu_1017[7] = new Lagu_2511531017("Happier", "Olivia Rodrigo",
175);
dataLagu_1017[8] = new Lagu_2511531017("Counting Stars",
"OneRepublic", 257);
dataLagu_1017[9] = new Lagu_2511531017("Stay", "The Kid Laroi", 141);
dataLagu_1017[10] = new Lagu_2511531017("Yellow", "Coldplay", 269);
dataLagu_1017[11] = new Lagu_2511531017("Faded", "Alan Walker", 212);
dataLagu_1017[12] = new Lagu_2511531017("Unstoppable", "Sia", 217);
jumlahData_1017 = 13;
}

public static void shellSort_1017() {
    int n_1017 = jumlahData_1017;
    int gap_1017 = n_1017 / 2;
    while (gap_1017 > 0) {
        for (int i_1017 = gap_1017; i_1017 < n_1017; i_1017++) {
            Lagu_2511531017 temp_1017 = dataLagu_1017[i_1017];
            int j_1017 = i_1017;
            while (j_1017 >= gap_1017 &&
                dataLagu_1017[j_1017 - gap_1017].judul_1017
                    .compareToIgnoreCase(temp_1017.judul_1017) > 0) {
                dataLagu_1017[j_1017] = dataLagu_1017[j_1017 - gap_1017];
                j_1017 = j_1017 - gap_1017;
            }
            dataLagu_1017[j_1017] = temp_1017;
        }
        gap_1017 = gap_1017 / 2;
    }
}

public static void tampilData_1017() {
    for (int i_1017 = 0; i_1017 < jumlahData_1017; i_1017++) {
        System.out.println((i_1017 + 1) + ". " +
dataLagu_1017[i_1017].judul_1017+ " | " + dataLagu_1017[i_1017].penyanyi_1017
+ " | " + dataLagu_1017[i_1017].durasi_1017+ " detik");
    }
}

public static void main(String[] args) {
    inputData_1017();
    System.out.println("=== DATA SEBELUM SORTING ===");
    tampilData_1017();
    shellSort_1017();
    System.out.println("\n=== DATA SETELAH SHELL SORT ===");
    tampilData_1017();
}
}

```

Kode 2.6

Penjelasan:

```
public static void inputData_1017() {  
    dataLagu_1017[0] = new Lagu_2511531017("Perfect", "Ed Sheeran", 263);  
    dataLagu_1017[1] = new Lagu_2511531017("Attention", "Charlie Puth",  
211);  
    dataLagu_1017[2] = new Lagu_2511531017("Believer", "Imagine Dragons",  
204);  
    dataLagu_1017[3] = new Lagu_2511531017("Shape Of You", "Ed Sheeran",  
240);  
    dataLagu_1017[4] = new Lagu_2511531017("Dandelions", "Ruth B", 233);  
    dataLagu_1017[5] = new Lagu_2511531017("Memories", "Maroon 5", 189);  
    dataLagu_1017[6] = new Lagu_2511531017("Photograph", "Ed Sheeran",  
258);  
    dataLagu_1017[7] = new Lagu_2511531017("Happier", "Olivia Rodrigo",  
175);  
    dataLagu_1017[8] = new Lagu_2511531017("Counting Stars",  
"OneRepublic", 257);  
    dataLagu_1017[9] = new Lagu_2511531017("Stay", "The Kid Laroi", 141);  
    dataLagu_1017[10] = new Lagu_2511531017("Yellow", "Coldplay", 269);  
    dataLagu_1017[11] = new Lagu_2511531017("Faded", "Alan Walker", 212);  
    dataLagu_1017[12] = new Lagu_2511531017("Unstoppable", "Sia", 217);  
    jumlahData_1017 = 13;  
}
```

Membuat dan menyimpan 13 objek lagu ke dalam array. Setiap objek lagu berisi informasi judul, penyanyi, dan durasi lagu. Setelah seluruh data dimasukkan, variabel jumlahData_1017 diisi dengan nilai 13 sebagai jumlah data yang tersedia.

```
public static void shellSort_1017() {  
    int n_1017 = jumlahData_1017;  
    int gap_1017 = n_1017 / 2;  
    while (gap_1017 > 0) {
```

Mengurutkan data lagu menggunakan algoritma Shell Sort. Variabel n_1017 digunakan untuk menyimpan jumlah data yang akan diurutkan. Variabel gap_1017 digunakan sebagai jarak awal antar elemen yang akan dibandingkan. Perulangan digunakan untuk menjalankan proses sorting selama nilai gap masih lebih besar dari nol. Nilai gap akan terus diperkecil setelah setiap tahap pengurutan selesai dilakukan. Perulangan berhenti ketika gap bernilai nol dan data sudah terurut.

```
for (int i_1017 = gap_1017; i_1017 < n_1017; i_1017++) {  
    Lagu_2511531017 temp_1017 = dataLagu_1017[i_1017];  
    int j_1017 = i_1017;
```

Menelusuri setiap data lagu berdasarkan nilai gap yang sedang digunakan. Lagu yang sedang diproses disimpan sementara ke dalam variabel temp_1017. Variabel j_1017 digunakan untuk mencari posisi yang tepat bagi lagu tersebut.

```
while (j_1017 >= gap_1017 && dataLagu_1017[j_1017 -  
gap_1017].judul_1017.compareToIgnoreCase(temp_1017.judul_1017) > 0) {
```

Membandingkan judul lagu secara alfabetis. Method compareToIgnoreCase() digunakan agar huruf besar dan huruf kecil dianggap sama saat proses perbandingan. Jika judul lagu sebelumnya lebih besar, maka data akan digeser ke posisi berikutnya.

```
dataLagu_1017[j_1017] = dataLagu_1017[j_1017 - gap_1017];
j_1017 = j_1017 - gap_1017;
```

Menggeser data lagu yang memiliki urutan lebih besar ke posisi kanan. Pergeseran dilakukan agar tersedia ruang untuk menempatkan lagu yang sedang diproses. Nilai indeks j_1017 kemudian dipindahkan mundur sesuai nilai gap.

```
dataLagu_1017[j_1017] = temp_1017;
}
gap_1017 = gap_1017 / 2;
```

Menempatkan lagu yang disimpan pada variabel sementara ke posisi yang sesuai dan memperkecil nilai gap setelah satu tahap sorting selesai dilakukan. Semakin kecil nilai gap, maka proses pengurutan menjadi semakin detail. Ketika gap bernilai satu, proses yang dilakukan mirip dengan Insertion Sort.

```
public static void tampilData_1017() {
    for (int i_1017 = 0; i_1017 < jumlahData_1017; i_1017++) {
        System.out.println((i_1017 + 1) + ". " +
            dataLagu_1017[i_1017].judul_1017+ " | " + dataLagu_1017[i_1017].penyanyi_1017
            + " | " + dataLagu_1017[i_1017].durasi_1017+ " detik");
    }
}
```

Mencetak seluruh data lagu ke layar. Informasi yang ditampilkan meliputi nomor urut, judul lagu, nama penyanyi, dan durasi lagu. Perulangan digunakan agar seluruh data dapat ditampilkan satu per satu.

```
public static void main(String[] args) {
    inputData_1017();
    System.out.println("=== DATA SEBELUM SORTING ===");
    tampilData_1017();
    shellSort_1017();
    System.out.println("\n=== DATA SETELAH SHELL SORT ===");
    tampilData_1017();
}
```

Menjalankan alur program secara berurutan. Program akan menampilkan data sebelum sorting, mengurutkan data menggunakan Shell Sort, lalu menampilkan hasil setelah sorting. Dengan cara ini pengguna dapat melihat perbedaan urutan data sebelum dan sesudah pengurutan.

Output:

```
Console X
<terminated> Sorting_2511531017 [Java Application] C:\Users\SISI\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_
=== DATA SEBELUM SORTING ===
1. Perfect | Ed Sheeran | 263 detik
2. Attention | Charlie Puth | 211 detik
3. Believer | Imagine Dragons | 204 detik
4. Shape Of You | Ed Sheeran | 240 detik
5. Dandelions | Ruth B | 233 detik
6. Memories | Maroon 5 | 189 detik
7. Photograph | Ed Sheeran | 258 detik
8. Happier | Olivia Rodrigo | 175 detik
9. Counting Stars | OneRepublic | 257 detik
10. Stay | The Kid Laroi | 141 detik
11. Yellow | Coldplay | 269 detik
12. Faded | Alan Walker | 212 detik
13. Unstoppable | Sia | 217 detik

=== DATA SETELAH SHELL SORT ===
1. Attention | Charlie Puth | 211 detik
2. Believer | Imagine Dragons | 204 detik
3. Counting Stars | OneRepublic | 257 detik
4. Dandelions | Ruth B | 233 detik
5. Faded | Alan Walker | 212 detik
6. Happier | Olivia Rodrigo | 175 detik
7. Memories | Maroon 5 | 189 detik
8. Perfect | Ed Sheeran | 263 detik
9. Photograph | Ed Sheeran | 258 detik
10. Shape Of You | Ed Sheeran | 240 detik
11. Stay | The Kid Laroi | 141 detik
12. Unstoppable | Sia | 217 detik
13. Yellow | Coldplay | 269 detik
```

Gambar 2.6

BAB III

KESIMPULAN

3.1 Kesimpulan

Sorting merupakan teknik yang digunakan untuk mengurutkan data agar lebih terstruktur dan mudah dikelola. Berbagai algoritma sorting memiliki mekanisme yang berbeda dalam melakukan proses pengurutan, namun tetap menghasilkan data yang terurut sesuai kriteria yang ditentukan.

Shell Sort melakukan pengurutan dengan memanfaatkan nilai gap yang terus diperkecil hingga data menjadi terurut. Quick Sort menggunakan metode divide and conquer dengan memilih pivot untuk membagi data menjadi beberapa bagian, sedangkan Merge Sort membagi data menjadi bagian yang lebih kecil kemudian menggabungkannya kembali dalam keadaan terurut.

Pemilihan algoritma sorting yang tepat dapat memengaruhi efisiensi proses pengolahan data. Oleh karena itu, pemahaman terhadap cara kerja dan karakteristik setiap algoritma sangat penting dalam pengembangan program dan pengolahan data yang lebih optimal.

3.2 Saran

Sebagai saran, sebaiknya penjelasan materi saat praktikum bisa lebih detail dan perlahan, supaya mahasiswa yang belum terlalu paham coding bisa mengikuti dengan baik.

DAFTAR PUSTAKA

- [1] Oracle Corporation, “The Java™ Tutorials,” Oracle Documentation. [Online]. Available: <https://docs.oracle.com/javase/tutorial/>
- [2] Oracle Corporation, “Collections Framework,” Oracle Documentation. [Online]. Available: <https://docs.oracle.com/javase/tutorial/collections/>